



RAPPORT INTERMÉDIAIRE

Dossier d'Analyse

Sujet :

API de calcul de DJU

Groupe 12 :

Aziz Yousfi

Eudes Sterlin

Gabriel Dahan

Gwenaël Lassalle

Marceau Feral

Tuteur :

Thierry Mathé

Table des matières

Introduction	2
1 Organisation	3
1.1 Cahier des charges	3
1.2 Questionnement et choix	3
1.3 Description des fonctionnalités	4
1.4 Gestion des données manquantes	5
1.5 Organisation du travail	5
2 Modélisation UML	6
2.1 Diagramme des cas d'utilisation	6
2.2 Diagramme d'activité	7
2.3 Diagramme de classe	9
2.4 Diagramme de séquence	10
2.5 Diagramme de packages	12
3 Fonctionnalités de l'API	14
3.1 Architecture générale	14
3.2 Accès public	14
3.3 Espace utilisateur authentifié	14
3.4 Persistance et mise en forme des résultats	14
3.5 Synthèse	14
Conclusion Intermédiaire	16
A Annexes	17
A.1 Diagrammes d'activité complémentaires	17
A.2 Diagrammes de séquence complémentaires	19

Introduction

Dans les secteurs résidentiel et tertiaire, les consommations d'énergie dépendent beaucoup de la météo : on chauffe davantage quand il fait froid, et on climatise davantage quand il fait chaud. Ainsi, lorsque l'on compare les consommations d'une année sur l'autre, il n'est pas toujours facile de savoir si une baisse vient d'une amélioration réelle (travaux, changements d'habitudes) ou simplement d'un hiver plus doux : il est difficile de différencier l'impact des variations de température de celui d'autres facteurs externes.

Pour tenir compte de cet effet, on utilise souvent un indicateur appelé Degré Jour Unifié (DJU). L'idée est assez simple : on mesure, jour après jour, l'écart entre la température extérieure et un seuil de confort thermique donné. Plus le climat est rigoureux (donc avec des conditions atmosphériques particulièrement dures), plus les DJU sont élevés. Cela permet ensuite de comparer des consommations en corrigeant, au moins en partie, l'influence de la météo.

Notre projet a pour but de concevoir et de développer une application sous forme d'API performante et modulaire, capable de calculer et d'agréger automatiquement les DJU. L'utilisateur doit pouvoir interroger le système aussi bien sur un point précis repéré par ses coordonnées géographiques (via une méthode d'interpolation à partir des stations Météo-France) que sur des découpages territoriaux administratifs (communes, départements, régions) ou sur des zonages personnalisés.

Dans un premier temps, nous détaillerons le cahier des charges, en précisant le périmètre du système, les besoins fonctionnels et les contraintes techniques. Dans un second temps, nous présenterons la conception générale et l'architecture logicielle de notre application à travers les différents diagrammes de modélisation et la structuration des bases de données. Finalement, nous décrirons les fonctionnalités attendues d'un point de vue utilisateur, avant de conclure sur cette note d'étape.

1 Organisation

Cette partie est consacrée au pilotage du projet. Elle formalise les spécifications du cahier des charges, justifie les choix fonctionnels de l’application et présente l’organisation du travail en groupe.

1.1 Cahier des charges

Fonctionnalités de base

Les fonctionnalités minimales de l’application sont :

- La création des bases de données à partir des relevés de température des stations de Météo France et du référentiel des communes disponibles sur data.gouv.fr
- Le calcul des DJU localisés sur une période et une périodicité donnée. Par exemple calcul des DJU mensuels du 1er janvier 2010 au 31 décembre 2025.
- Intégrer les zonages Département, Région pour tous les utilisateurs
- Le calcul des DJU sur un territoire avec une granularité, une période et une périodicité données. Par exemple calculer les DJU annuels des départements d’Ile-de-France du 1er janvier.
- Permettre à un utilisateur authentifié de créer et de conserver ses propres zonages constitués d’ensemble de commune.

Fonctionnalités optionnelles

Les fonctionnalités optionnelles de l’application sont :

- Conserver les résultats des calculs effectués
- Mise en forme des résultats sous différents formats
- Importer des zonages à partir de fichiers csv, json

1.2 Questionnement et choix

La mise en place de l’architecture et du moteur de calcul a nécessité de répondre à plusieurs problématiques techniques et méthodologiques dès la phase de conception. Celles-ci concernent l’authentification des utilisateurs, le stockage et le format des données massives, les méthodes géométriques d’estimation des températures, ainsi que les stratégies de mise en cache.

Le premier questionnement a porté sur la gestion des droits et l’accès à l’API : fallait-il concevoir une application totalement ouverte ou intégrer un système de comptes authentifiés ? Contrairement aux fonctionnalités de consultation pure des DJU (ponctuels ou par divisions administratives standard comme les départements et régions) qui doivent rester accessibles de manière directe, le cahier des charges impose de permettre à un utilisateur de sauvegarder ses propres découpages territoriaux. Nous avons donc arbitré en faveur d’un système à double niveau :

- Un accès public (“visiteur”) : permettant d’effectuer un calcul ponctuel (coordonnées GPS) ou territorial standard sans contrainte d’authentification.
- Un accès authentifié (“utilisateur enregistré”) : permettant à l’utilisateur de créer, consulter, modifier et supprimer ses zonages personnalisés sans interférer avec les données des autres usagers.

Le second questionnement a concerné l’ingestion et la persistance des relevés de température issus de Météo-France. Les fichiers sources départementaux représentent un volume considérable de données compressées au format CSV. L’utilisation directe de ces fichiers bruts à chaque requête aurait provoqué des temps de latence.

Pour les données de référence géographiques et la gestion des comptes/zonages utilisateurs, nous avons opté pour une base de données relationnelle SQL (accessible via notre couche DAO), garantissant l'intégrité référentielle des communes et des utilisateurs. Pour les séries temporelles de températures, la conversion vers un format analytique optimisé (fichiers Parquet indexés par station et par date) a été privilégiée pour accélérer les opérations de filtrage temporel et de jointures spatiales.

De plus, ces jeux de données comportent des valeurs manquantes. Nous avons décidé d'appliquer une interpolation temporelle locale sur les relevés immédiatement voisins pour combler des absences ponctuelles quand cela sera possible.

Il est rare de disposer d'une station météo sur place pour évaluer la température en un point donné. Le calcul nécessite donc d'interpoler les valeurs mesurées par les stations environnantes.

Les coordonnées étant fournies sous forme de latitude et longitude, nous utiliserons la méthode d'Haversine pour calculer les distances. Ensuite, la méthode de pondération inverse des distances a été choisie pour affecter un poids prépondérant aux stations les plus proches. Enfin, pour respecter la réalité physique du climat, nous avons intégré la correction altimétrique standard préconisée, consistant à soustraire 0,65 °C par tranche de 100 mètres d'élévation entre les stations et le point cible.

Enfin, le dernier point critique concerne le coût en calcul des agrégations géographiques. Calculer les DJU d'une région ou d'un département implique d'estimer les DJU de chaque commune membre, puis de pondérer ces valeurs par la population communale respective. Répéter cette série de calculs pour des requêtes identiques sur les mêmes périodes aurait saturé le serveur. Nous avons donc fait le choix de concevoir un mécanisme de cache et de persistance des résultats intermédiaires : les températures quotidiennes interpolées par commune ainsi que les agrégations annuelles ou mensuelles standard sont conservées après leur premier calcul, transformant les requêtes ultérieures en de simples lectures optimisées en base de données.

1.3 Description des fonctionnalités

La première fonctionnalité de notre application consiste en l'ingestion et la structuration des données sources nécessaires aux calculs. L'application doit pouvoir charger, nettoyer et intégrer deux grands ensembles de données ouverts issus de data.gouv.fr : d'une part, les relevés climatologiques quotidiens des stations de Météo-France (températures minimales, maximales, coordonnées géographiques et altitudes), et d'autre part, le référentiel administratif et démographique des communes françaises. Cette étape préliminaire constitue le socle indispensable au fonctionnement du moteur de calcul.

Sur cette base, le cœur de l'application repose sur le calcul des DJU ponctuels. À partir de coordonnées géographiques (latitude, longitude, altitude), le système évalue la température locale via une interpolation spatiale par pondération inverse des distances prenant en compte les stations météo environnantes. L'utilisateur peut alors paramétrer les seuils de confort thermique souhaités (par exemple 15 °C ou 18 °C pour le chauffage, et un seuil dédié pour la climatisation) ainsi que l'horizon temporel (dates de début et de fin) et le pas d'agrégation temporelle (cumuls quotidiens, mensuels ou annuels).

L'application propose également une dimension d'agrégation territoriale. En effet, l'utilisateur doit pouvoir obtenir les DJU d'un territoire administratif complet (départements, régions ou France métropolitaine). Le calcul ne s'effectue plus ici en un point unique, mais nécessite de calculer les DJU de l'ensemble des communes composant le territoire visé, puis de réaliser une agrégation spatiale par moyenne pondérée par la population de chaque commune. Cela permet d'obtenir un indicateur représentatif de l'exposition réelle des populations aux variations climatiques.

Enfin, pour offrir un service modulable et adapté aux besoins métiers, un espace utilisateur authentifié est intégré. Il permet à un utilisateur de concevoir, sauvegarder et administrer ses propres zonages territoriaux sur mesure (regroupements spécifiques de communes, territoires de

contrats de transition écologique, zones d'études privées), afin de pouvoir relancer des calculs récurrents sur ces périmètres sans avoir à redéfinir la liste des communes à chaque requête.

1.4 Gestion des données manquantes

Le traitement des données manquantes est un sujet important du projet car les fichiers Météo-France bruts comportent parfois d'importantes périodes sans données. Tout d'abord, les antennes couvrant moins de 65 % de la période d'intérêt sont écartées tout comme les doublons spatiaux distants de moins de 500 mètres (on conserve la plus complète). On détecte aussi les anomalies évidentes telle que des températures constante sur 30 jours ou des valeurs extrêmes physiquement impossibles et on élimine ces séries.

Pour le comblement des trous temporels, la stratégie s'affine selon la durée du gap et repose sur l'exploitation combinée de l'information locale et spatiale. Pour les absences de 1 à 2 jours, interpolation linéaire simple. Entre 3 et 7 jours, on fait une interpolation hybride : on calcule d'abord une interpolation linéaire basique (tendance locale entre $j-1$ et $j+1$), puis simultanément on récupère les observations réelles des 3 à 5 antennes spatiales les plus proches sur la même période. On fusionne les deux sources en pondérant 70 % la tendance locale et 30 % la moyenne des antennes voisines, ce qui donne une courbe moins rigide qu'une droite tout en restant ancrée dans la variabilité régionale réelle. Pour les gaps plus longs, de 7 à 30 jours, on utilise une régression linéaire multi-station : sur la période où les données de la station cible sont intactes, on estime une relation linéaire entre la station cible et ses voisines proches (typiquement $T_{\text{cible}} = a \times T_{\text{proche_A}} + b \times T_{\text{proche_B}} + c \times T_{\text{proche_C}}$). On fit les coefficients (a , b , c) sur les données complètes, ce qui capture comment la station co-varie réellement avec son environnement spatial. Pendant le gap, on applique cette relation aux observations réelles des antennes proches pour prédire la station cible. Optionnellement, si plusieurs années historiques sont disponibles, on peut ajouter une correction saisonnière en calculant l'anomalie moyenne (écart station-région) pour la même période calendaire les années antérieures. Les gaps dépassant 30 jours restent rejetés : au-delà, aucune méthode statistique fiable ne compense l'absence totale d'information.

Enfin, chaque résultat inclura un rapport de qualité : pourcentage de jours avec données originales, proportion de jours interpolés vs. spatialisés vs. rejetés. Ces métadonnées permettront à l'utilisateur de juger la fiabilité de son DJU. Cette méthodologie sera consolidée et potentiellement affinée lors de la mise en pratique de ces méthodes et de leurs tests.

1.5 Organisation du travail

L'application sera développée en Python. Nous utiliserons PostgreSQL pour gérer la base de données de l'application. Enfin, pour simplifier la coordination du développement, nous utilisons github.

...

2 Modélisation UML

2.1 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation identifie trois profils. L'utilisateur anonyme peut calculer un DJU ponctuel ou zonal et consulter les zonages officiels (départements et régions). L'utilisateur authentifié hérite de ces droits et peut en outre créer, importer et gérer des zonages personnalisés constitués d'ensembles de communes. L'administrateur est chargé d'alimenter l'application à partir des données sources (relevés Météo-France et référentiel géographique). L'import d'un fichier de zonage étend le cas d'utilisation de création d'un zonage personnalisé.

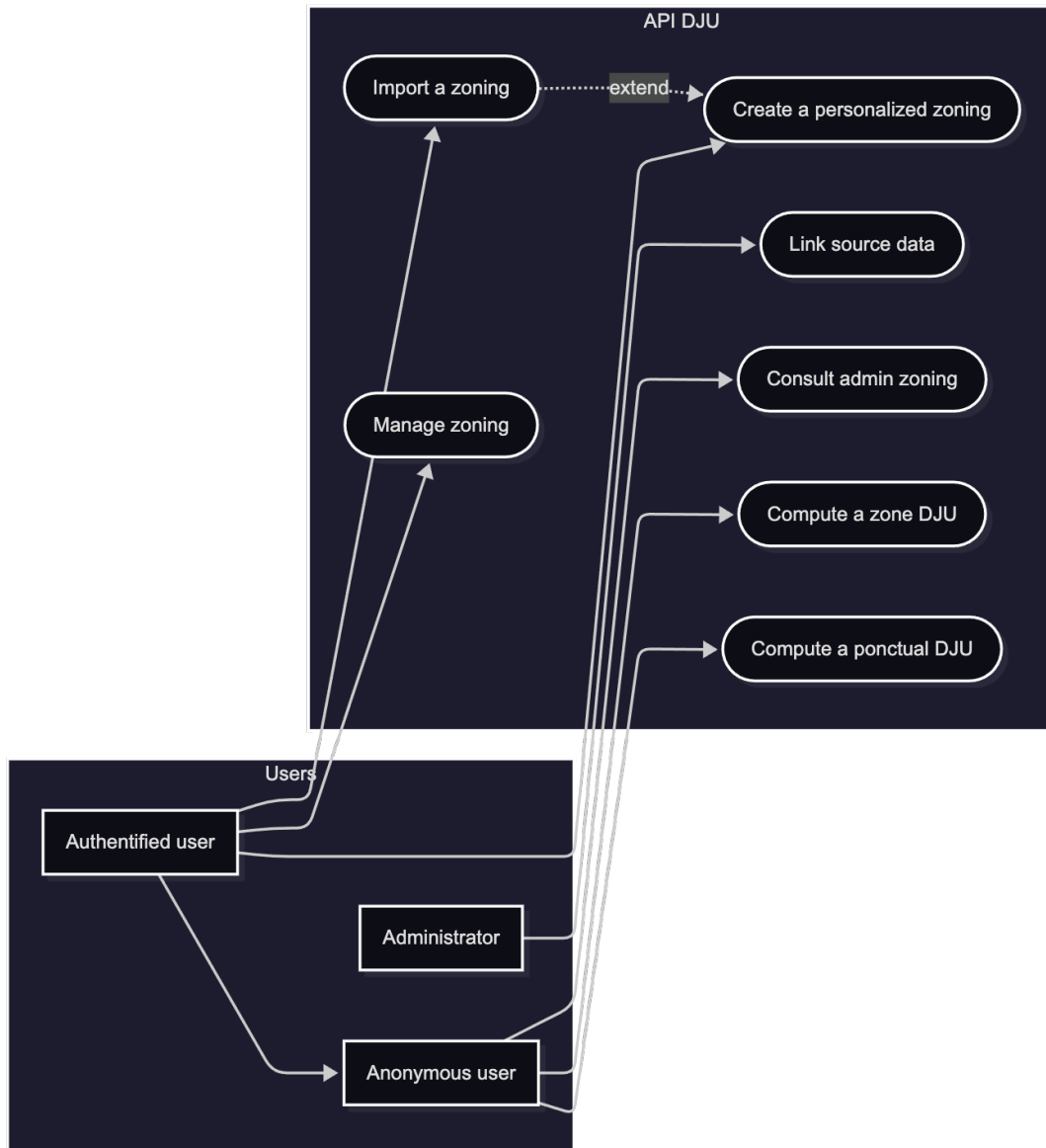


Figure 2.1 : Diagramme des cas d'utilisation

2.2 Diagramme d'activité

Les diagrammes d'activité décrivent l'enchaînement des traitements métier, sans entrer dans le détail des couches logicielles. Nous présentons ici les quatre flux centraux : initialisation des données, estimation des températures, calcul d'un DJU ponctuel et calcul d'un DJU zonal. Les diagrammes portant sur la consultation, la création, l'import et la gestion des zonages sont regroupés en annexe.

L'initialisation des bases de données (figure~2.2) charge les fichiers de températures Météo-France par département, filtre les colonnes utiles et les dates à partir de 1990, déduplique les relevés, puis enregistre les stations et les observations quotidiennes. Le référentiel des communes est ensuite téléchargé, enrichi en altitude et structuré en départements et régions. L'estimation des températures locales (figure~2.3) sélectionne les stations les plus proches, applique éventuellement une correction d'altitude, puis interpole les séries quotidiennes par pondération inverse de la distance (IDW).

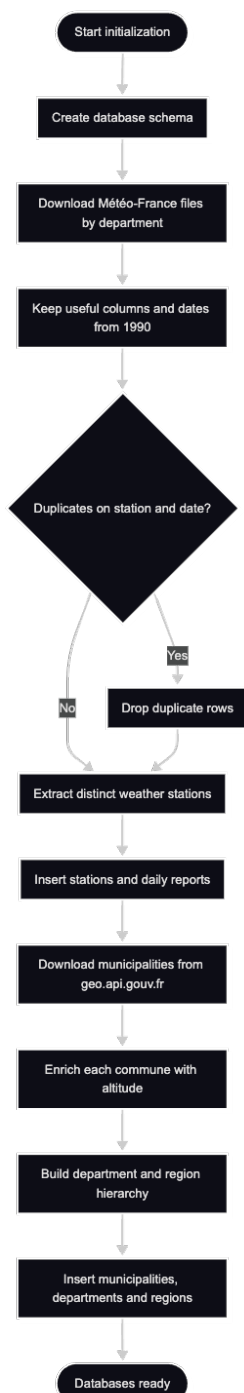


Figure 2.2 : Initialisation des bases de données

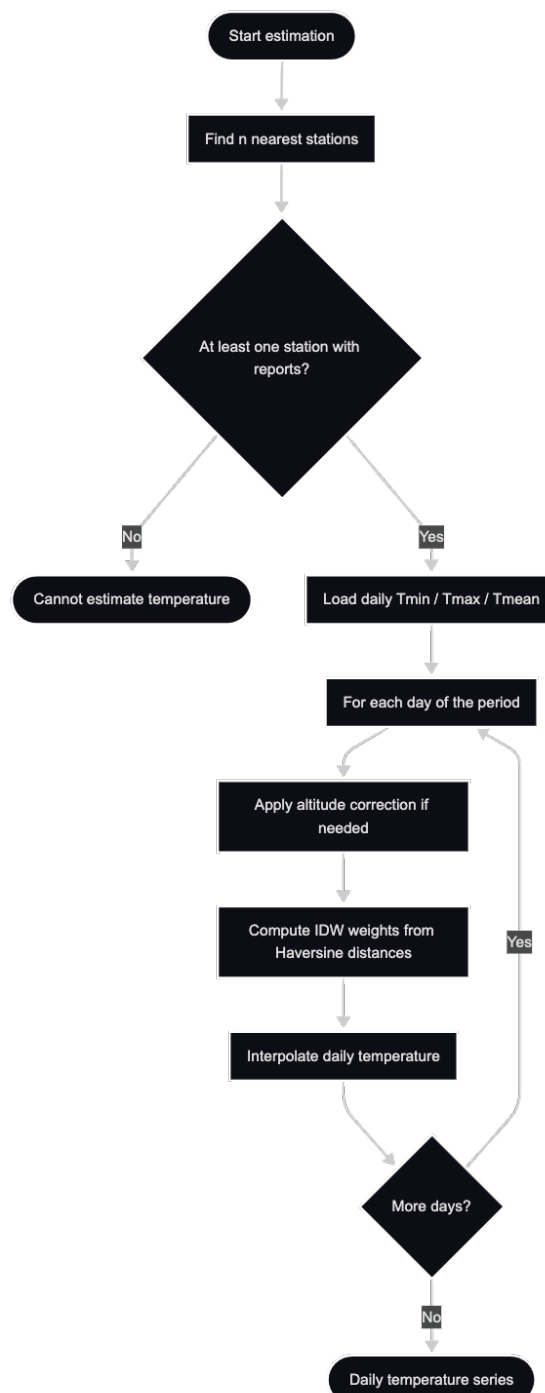


Figure 2.3 : Estimation des températures locales

Le calcul d'un DJU ponctuel (figure~2.4) part d'une requête sur des coordonnées GPS. Après validation des paramètres, un résultat déjà calculé peut être réutilisé ; sinon les températures quotidiennes sont estimées, les DJU de chauffage (et éventuellement de climatisation) sont obtenus jour par jour, puis agrégés selon le pas de temps demandé. Le calcul d'un DJU zonal (figure~2.5) charge d'abord le territoire et ses communes. Les zonages officiels sont publics, tandis qu'un zonage personnalisé n'est accessible qu'à son propriétaire. Les DJU communaux sont ensuite agrégés, par exemple en les pondérant par la population.

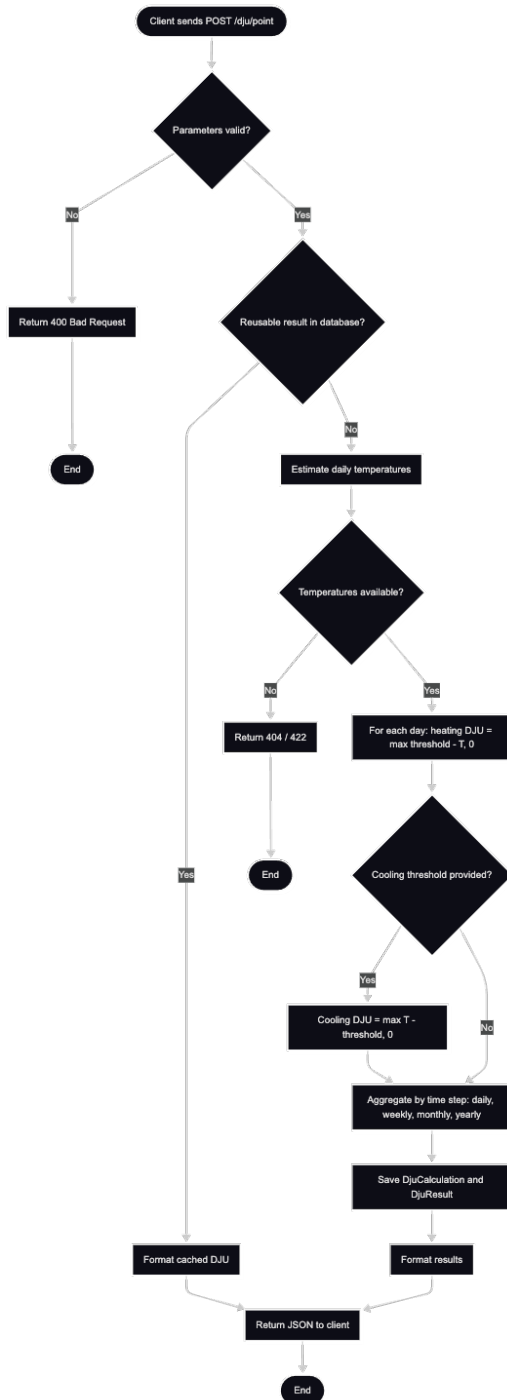


Figure 2.4 : Calcul d'un DJU ponctuel

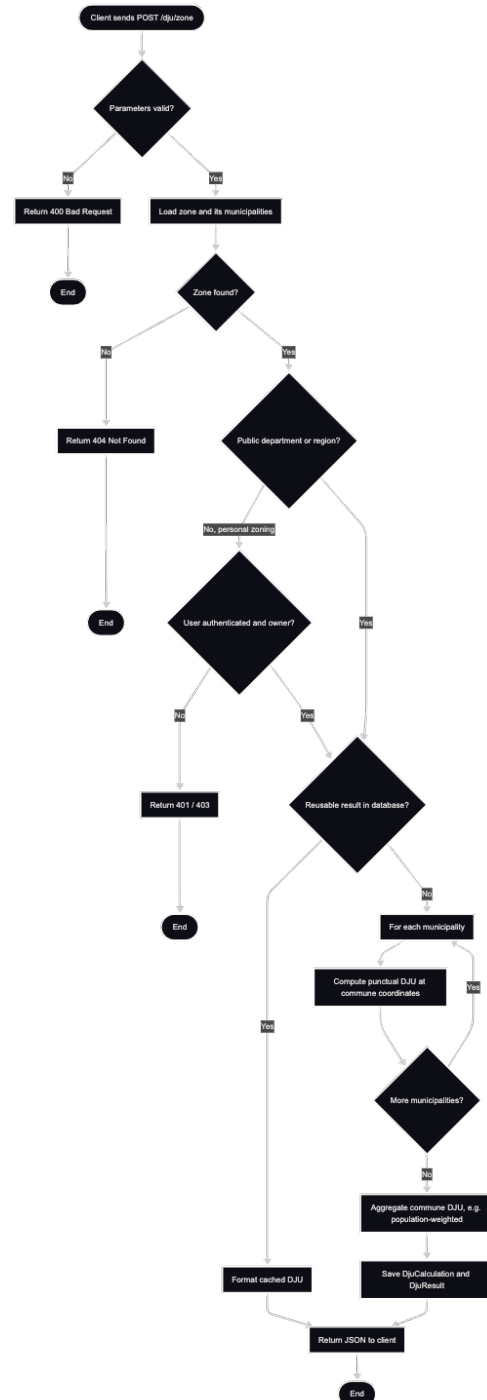


Figure 2.5 : Calcul d'un DJU zonal

2.3 Diagramme de classe

Le diagramme de classes structure le domaine de l'application. **GeographicZone** factorise les zonages officiels : une région agrège des départements, un département agrège des communes. **Zoning** représente un territoire personnalisé, ensemble de communes associé à un **User**. Les stations météorologiques et leurs relevés quotidiens (**MeteoStation**, **TemperatureReport**) alimentent un **DjuCalculation**, qui produit des **DjuResult** selon un **DjuType** (chauffage ou climatisation) et un pas de temps d'agrégation.

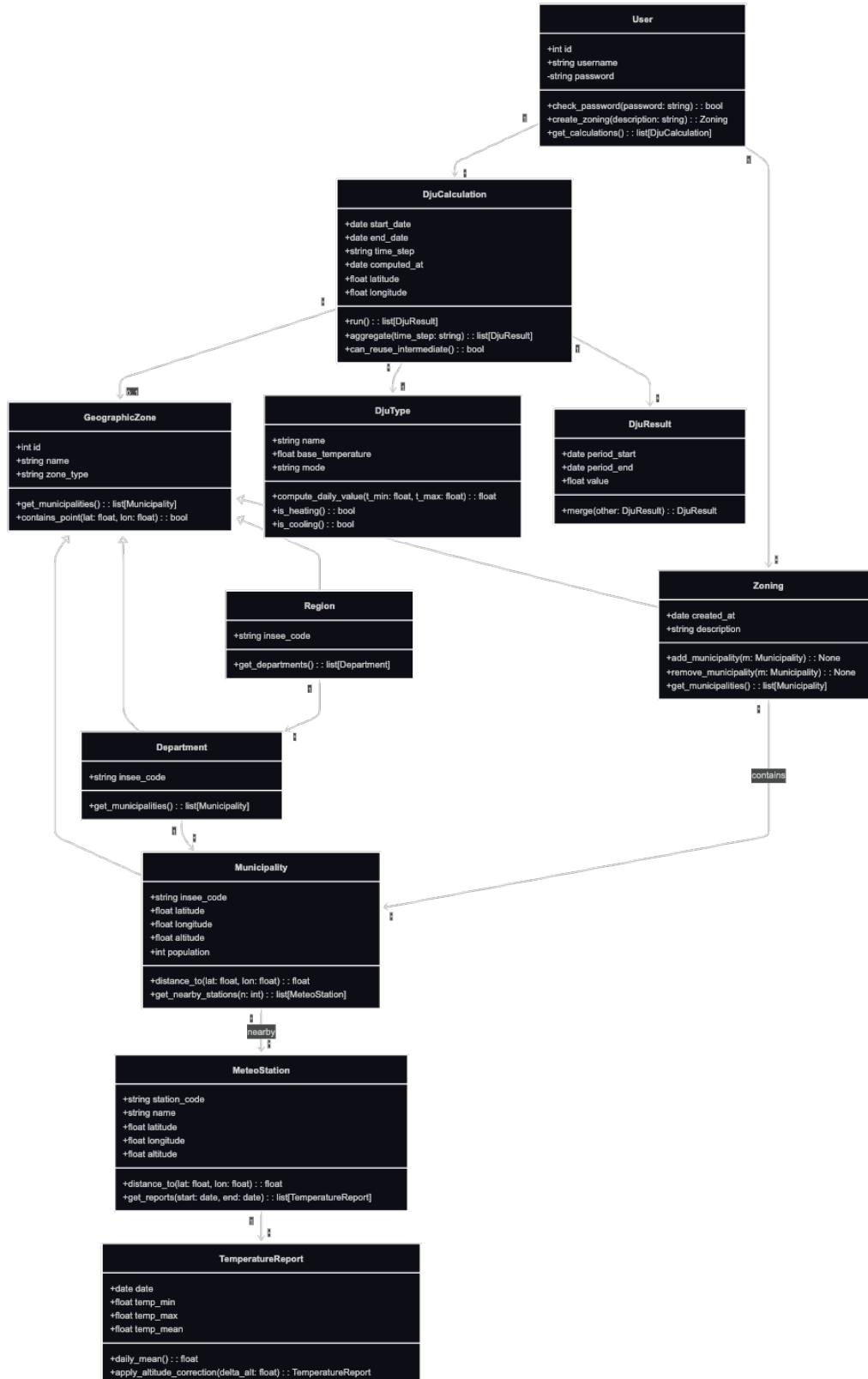


Figure 2.6 : Diagramme de classes

2.4 Diagramme de séquence

Les diagrammes de séquence résument les échanges entre le client, les couches de l'API (contrôleur, service, DAO) et la base de données. Nous détaillons ici les trois échanges centraux : initialisation des bases (F0), calcul d'un DJU ponctuel (F1) et calcul d'un DJU zonal (F3). Les autres diagrammes de séquence (consultation des zonages, authentification, création, gestion et import de zonages) figurent en annexe.

L'administrateur déclenche l'initialisation des bases (F0). Les scripts d'alimentation récupèrent les températures Météo-France et le référentiel géographique auprès des sources externes, puis les enregistrent via la couche DAO.

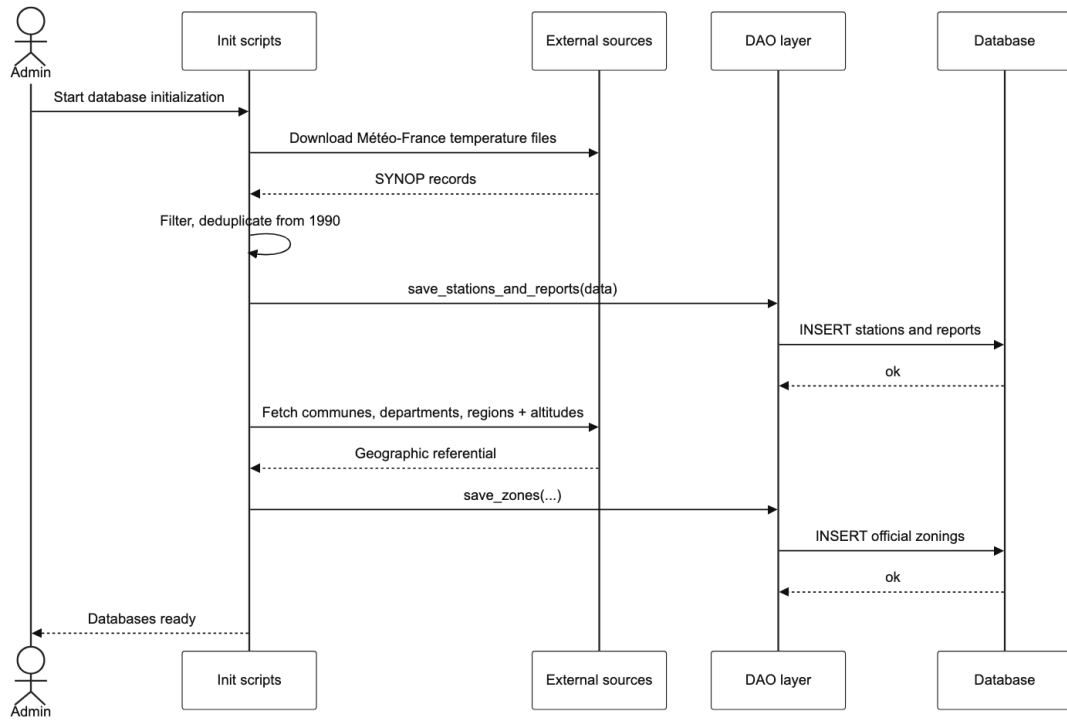


Figure 2.7 : Initialisation des bases de données (F0)

Un client demande un DJU ponctuel (F1). Si un résultat comparable est déjà en cache, il est renvoyé ; sinon le service estime les températures (IDW, correction d'altitude), calcule les DJU et les enregistre.

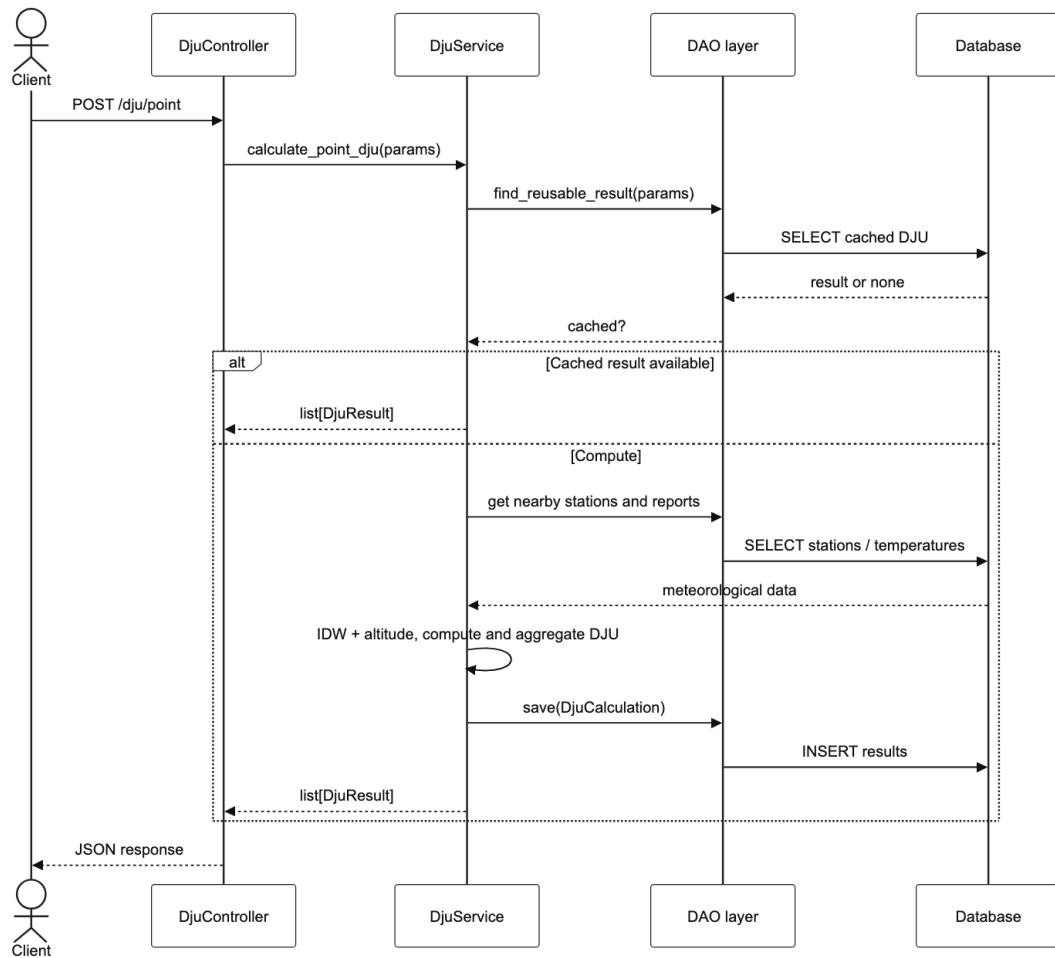


Figure 2.8 : Calcul d'un DJU ponctuel (F1)

Le calcul d'un DJU zonal (F3) s'applique à un département, une région ou un zonage personnalisé (authentification requise pour un zonage personnel). Les DJU communaux sont agrégés sur le territoire, avec réutilisation du cache lorsque possible.

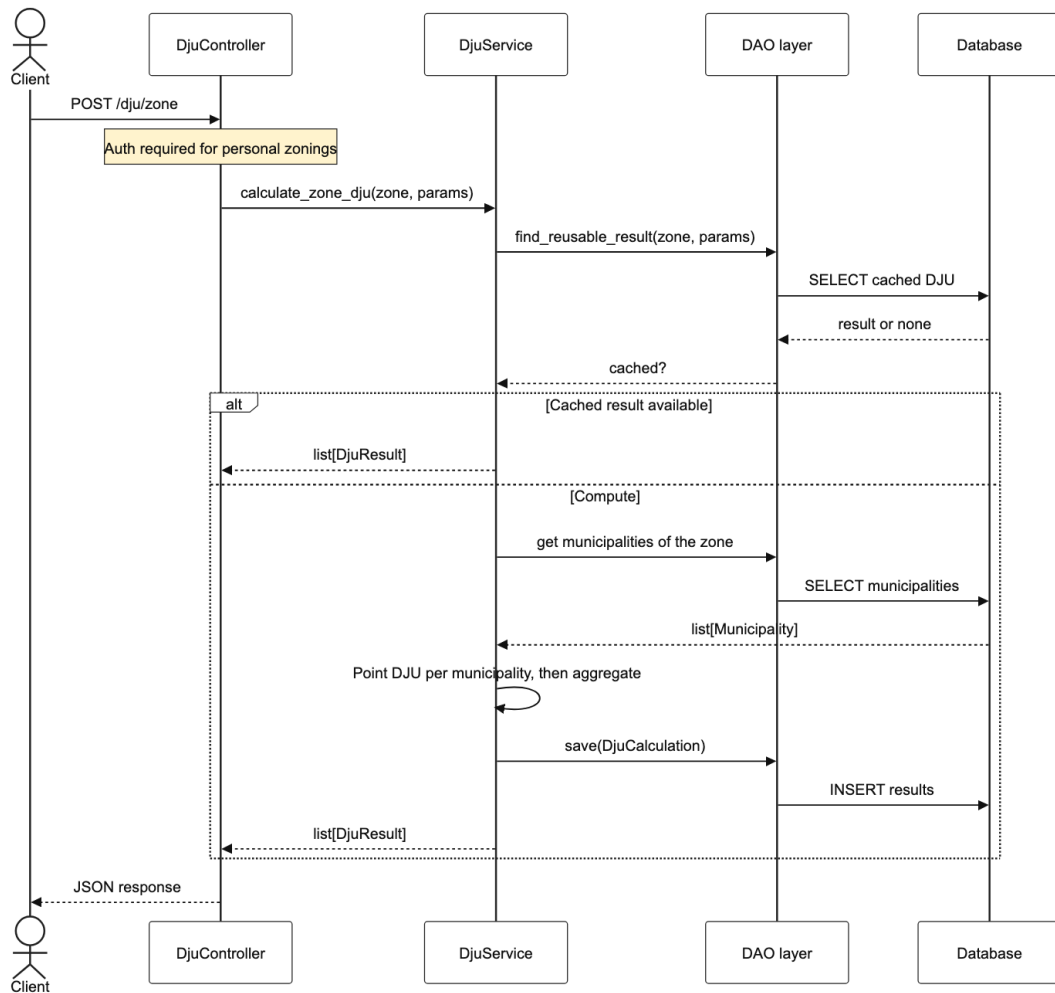


Figure 2.9 : Calcul d'un DJU zonal (F3)

2.5 Diagramme de packages

Afin de structurer efficacement notre projet, nous avons opté pour une architecture logicielle en couches qui garantit une nette séparation des responsabilités. La couche de présentation (Presentation Layer) expose l'API REST développée sous Python ; elle réceptionne les requêtes HTTP (calculs ponctuels, territoriaux ou gestion des zonages), valide les paramètres d'entrée et formate les réponses en JSON, CSV sans porter de logique métier. Elle transmet ces demandes à la couche de service (Service Layer), le cœur algorithmique de l'application, qui orchestre le calcul des DJU, l'interpolation spatiale IDW corrigée de l'altitude, l'authentification des utilisateurs et le contrôle préalable du cache pour éviter les recalculs redondants. Cette couche manipule des entités pivots définies dans la couche métier (Business Layer), telles que les stations météo, les communes ou les résultats de DJU. Enfin, les échanges avec les supports de persistance sont entièrement délégués à la couche d'accès aux données (DAO), qui abstrait à la fois l'interrogation de la base relationnelle SQL et la lecture des températures stockés aux formats Parquet. Enfin, un module de tests unitaires permet de vérifier la justesse de nos calculs et le bon fonctionnement de l'API.

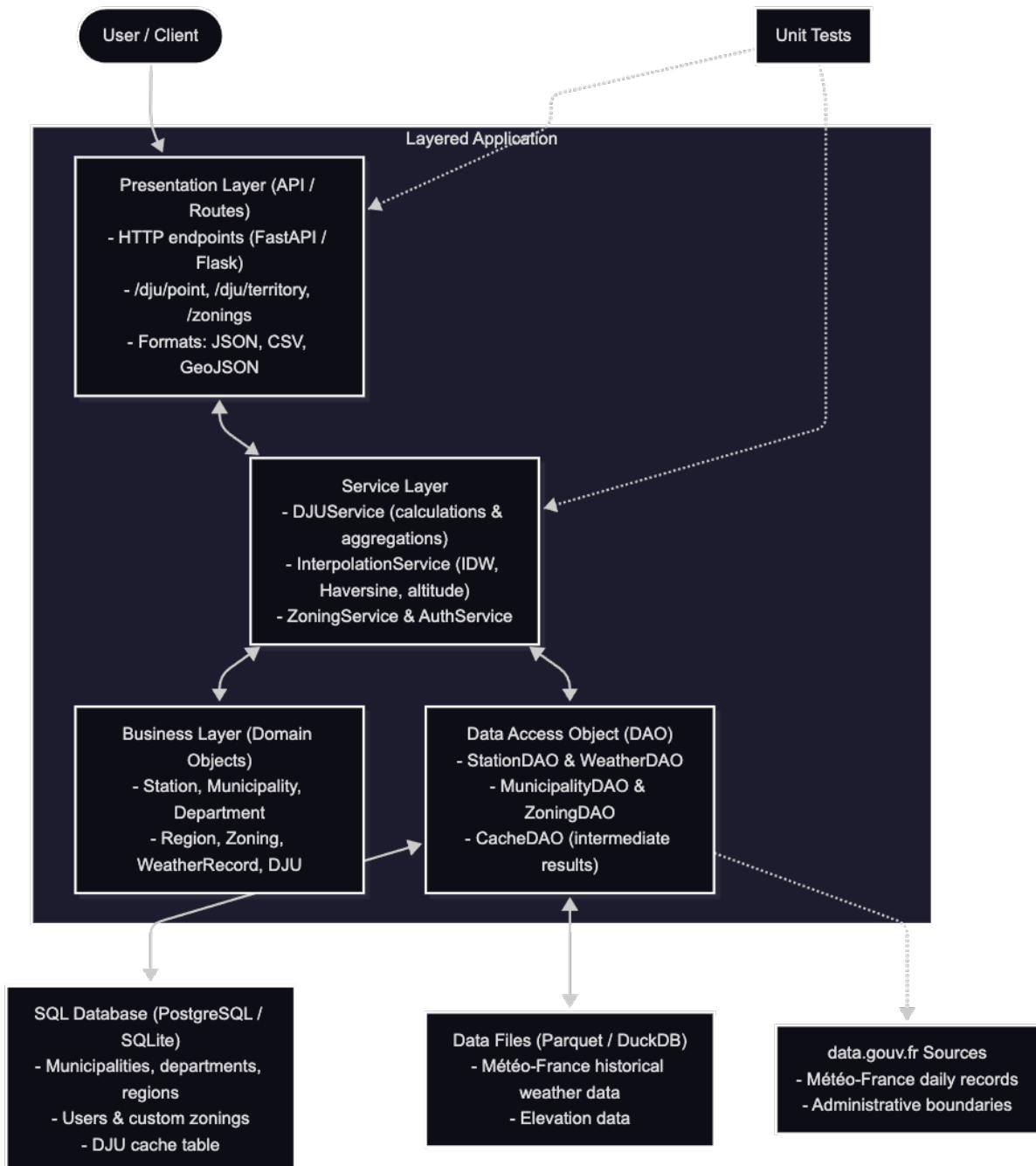


Figure 2.10 : Diagramme de packages

3 Fonctionnalités de l'API

Cette section présente les fonctionnalités exposées par l'API et la manière dont un client externe utilise ces fonctionnalités, décrites dans le cahier des charges.

3.1 Architecture générale

Chaque route HTTP est prise en charge par un contrôleur (`DjuController`, `ZoneController`, `UserController`), qui valide la requête puis délègue le traitement métier à une couche de services (`DjuService`, `ZoneService`, `AuthService`, `TemperatureService`), comme le montrent les diagrammes de séquence. Ces services orchestrent les objets métier du diagramme de classes : en particulier `DjuCalculation.run()`, qui pilote l'estimation des températures puis le calcul et l'agrégation des DJU, et `GeographicZone/Zoning`, qui expose la liste des communes d'un territoire via `get_municipalities()` et s'appuient sur la couche DAO pour la persistance.

Le fichier principal de l'application sert de point d'entrée à l'API : c'est lui qui enregistre les différents contrôleurs. Quand une requête HTTP arrive, on commence par valider les données reçues à l'aide de schémas dédiés, avant de les transmettre au contrôleur concerné. Ce dernier ne traite pas lui-même la logique métier, il la délègue à la couche de services, qui se charge des calculs et, si besoin, dialogue avec la couche d'accès aux données.

Cette organisation en couches distinctes (gestion des requêtes, validation, logique métier, persistance) rend le code plus facile à maintenir et à faire évoluer dans le temps.

3.2 Accès public

Sans authentification, un client peut :

- calculer un DJU ponctuel à partir de coordonnées GPS (F1) ;
- consulter les zonages officiels (départements et régions) et les communes qui les composent (F2) ;
- calculer un DJU sur un zonage officiel (F3).

3.3 Espace utilisateur authentifié

Après création d'un compte et authentification, un utilisateur accède en plus à la gestion de ses zonages personnalisés (F4) : création, consultation, modification et suppression, ainsi que le calcul de DJU sur ces zonages. Ces opérations s'appuient sur `User.create_zoning()` et sur les méthodes `add_municipality()` / `remove_municipality()` de `Zoning`. L'import d'un zonage depuis un fichier CSV ou JSON (FO3) suit le même chemin que la création manuelle, une fois les communes du fichier résolues.

3.4 Persistance et mise en forme des résultats

Conformément au choix de conception décrit en section 1.2, chaque calcul de DJU est transparent pour le client mais stocké de manière permanente côté serveur (FO1) : une requête portant sur des paramètres déjà calculés renvoie directement le résultat en base plutôt que de relancer l'estimation des températures et l'agrégation, capacité déjà présente au niveau métier via `DjuCalculation.can_reuse_intermediate()`. La mise en forme des résultats (FO2) est négociable au niveau du format de sortie (JSON par défaut), sans route dédiée distincte.

3.5 Synthèse

Fonct.	Route	Authentification	Description
F1	POST /dju/point	Optionnelle	Calcul DJU sur un point GPS
F2	GET /zones	Aucune	Liste des zonages officiels et de leurs communes
F3	POST /dju/zone	Optionnelle ¹	Calcul DJU sur un territoire
F4	POST /user/register	Aucune	Création d'un compte
F4	POST /user/login	Aucune	Authentification, émission d'une session
F4	GET /user/zones	Requise	Liste des zonages de l'utilisateur
F4	POST /user/zones	Requise	Création d'un zonage personnalisé
F4	PATCH /user/zones/{id}	Requise (propriétaire)	Modification d'un zonage
F4	DELETE /user/zones/{id}	Requise (propriétaire)	Suppression d'un zonage
FO1	—	—	Résultats mis en cache automatiquement, réutilisés par F1/F3
FO2	—	—	Format de sortie négociable (JSON par défaut)
FO3	POST /user/zones/import	Requise	Import d'un zonage depuis un fichier CSV/JSON

¹ Authentification obligatoire lorsque la zone ciblée est un zonage personnalisé.

Conclusion Intermédiaire

Ce rapport d'étape formalise les choix architecturaux et fonctionnels retenus pour développer une API de calcul de Degré Jour Unifié. La conception proposée satisfait l'ensemble du cahier des charges en proposant une double couche d'accès : une première sans authentification pour les consultations ponctuelles et les territoires standards, et une seconde réservée aux utilisateurs enregistrés pour la gestion de zonages personnalisés.

L'architecture en couches garantit une séparation claire des responsabilités entre présentation, orchestration métier et persistance. Le choix stratégique de dualiser le stockage : SQL pour les données de référence et géométriques, fichiers Parquet pour les séries temporelles volumineuses—répond aux contraintes de performance et d'évolutivité du projet. La mise en œuvre d'un mécanisme de cache et de persistance des résultats intermédiaires anticipe les demandes récurrentes et transforme les requêtes en lectures optimisées, prévenant ainsi une surcharge du serveur.

Les diagrammes de modélisation UML présentés esquissent le squelette logique du système. Les diagrammes de séquence détaillent les trois flux centraux (initialisation, calcul ponctuel, agrégation territoriale) et montrent comment la couche DAO abstrait les interactions avec les supports de persistance. Ces éléments structurent et clarifient les responsabilités de chaque module avant d'entrer dans la phase de développement itératif.

Les étapes suivantes porteront sur l'implémentation effective de ces composants : création des schémas de base de données, développement des services de calcul et d'interpolation spatiale, mise en place du système d'authentification, et mise en œuvre des tests unitaires et d'intégration. Une attention particulière sera accordée à la robustesse du traitement des données manquantes et à la validation de la qualité des résultats numériques pour assurer la fiabilité de l'indicateur de consommation énergétique. La phase de tests en charge permettra de vérifier que l'architecture scalabilise correctement aux volumes de données réels. Enfin, la documentation des endpoints API et la rédaction d'un guide utilisateur concluront ce cycle de développement et prépareront le déploiement en environnement de production.

A Annexes

A.1 Diagrammes d'activité complémentaires

Les diagrammes suivants complètent ceux du corps du rapport. Ils couvrent la consultation des zonages officiels, ainsi que la création, l'import et la gestion des zonages personnalisés.

La consultation des zonages (figure~A.1) permet de lister les départements, les régions ou l'ensemble des zonages officiels. La création ou l'import d'un zonage personnalisé (figure~A.2) exige une authentification : l'utilisateur fournit une liste de communes ou un fichier CSV / JSON, que l'application contrôle avant d'enregistrer le zonage.

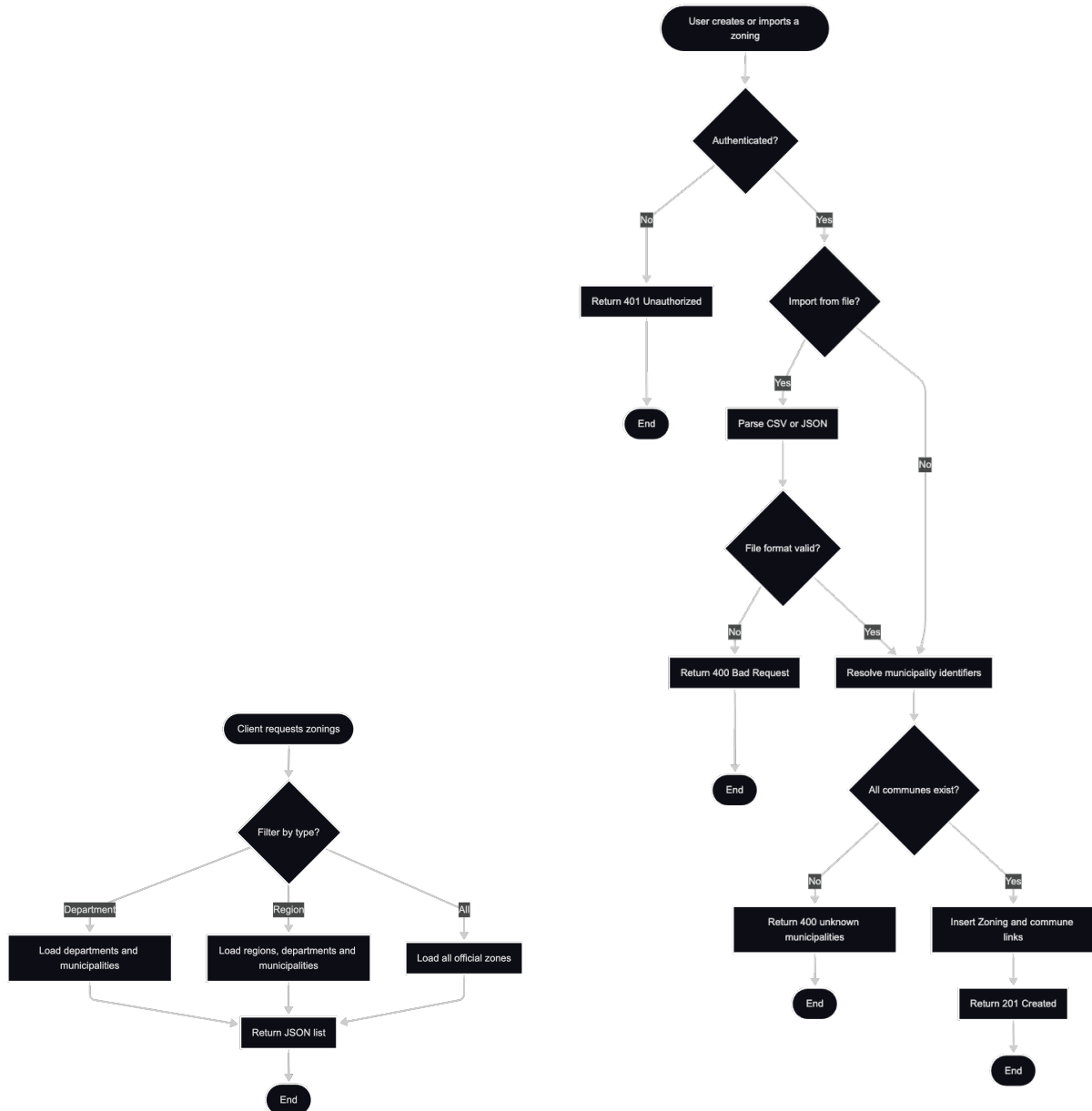


Figure A.1 : Consultation des zonages

Figure A.2 : Création ou import d'un zonage personnalisé

La gestion d'un zonage personnalisé (figure~A.3) est réservée au propriétaire. Après authentification, celui-ci peut modifier la description et la composition communale du zonage, ou le supprimer.

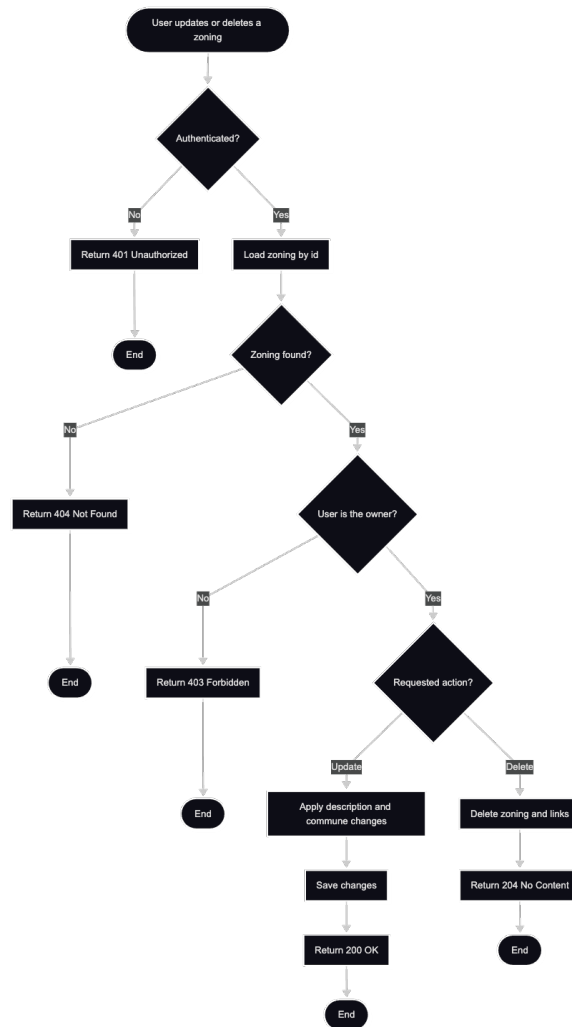


Figure A.3 : Gestion d'un zonage personnalisé

A.2 Diagrammes de séquence complémentaires

Les diagrammes suivants détaillent les échanges restants : consultation des zonages publics (F2), authentification, création et gestion d'un zonage personnalisé (F4), et import depuis un fichier (FO3).

Tout client peut consulter les zonages publics (F2) : départements et régions, ainsi que les communes qui les composent.

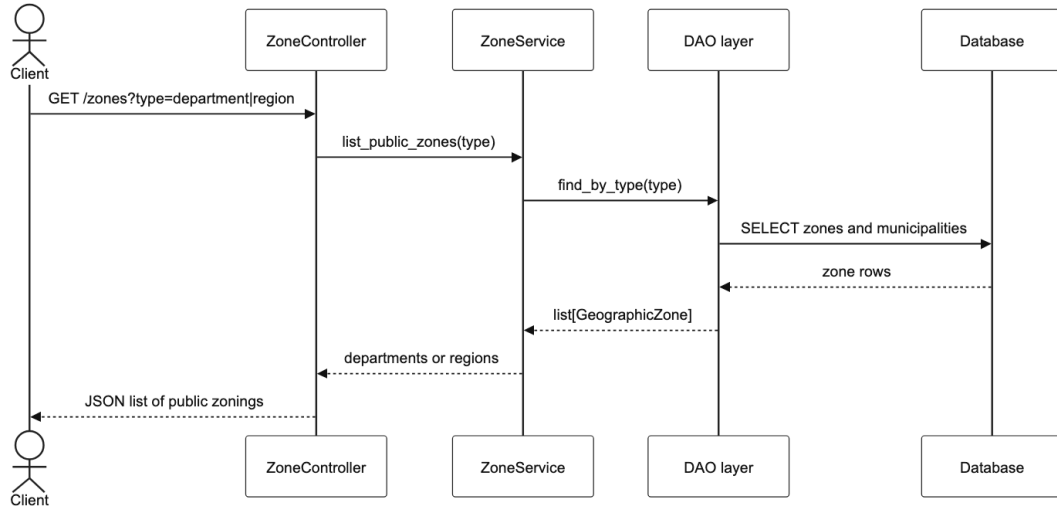


Figure A.4 : Consultation des zonages publics (F2)

L'authentification précède toute opération sur les zonages personnels (F4). L'API vérifie les identifiants, puis ouvre une session.

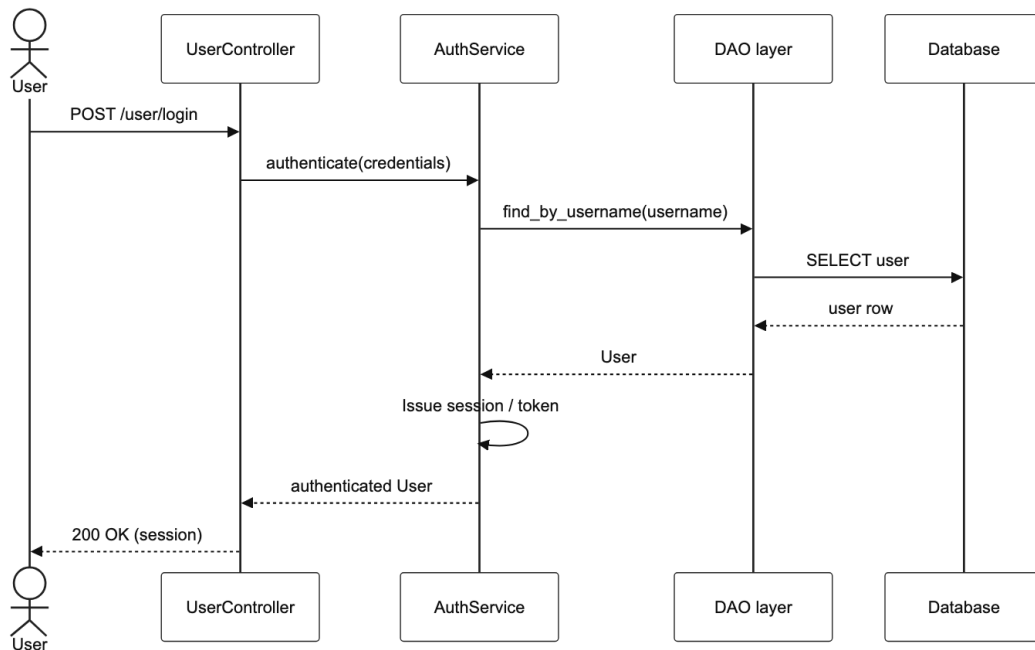


Figure A.5 : Authentification d'un utilisateur

Un utilisateur authentifié crée un zonage personnalisé (F4) en envoyant une description et une liste de communes. Après validation, le zonage est enregistré.

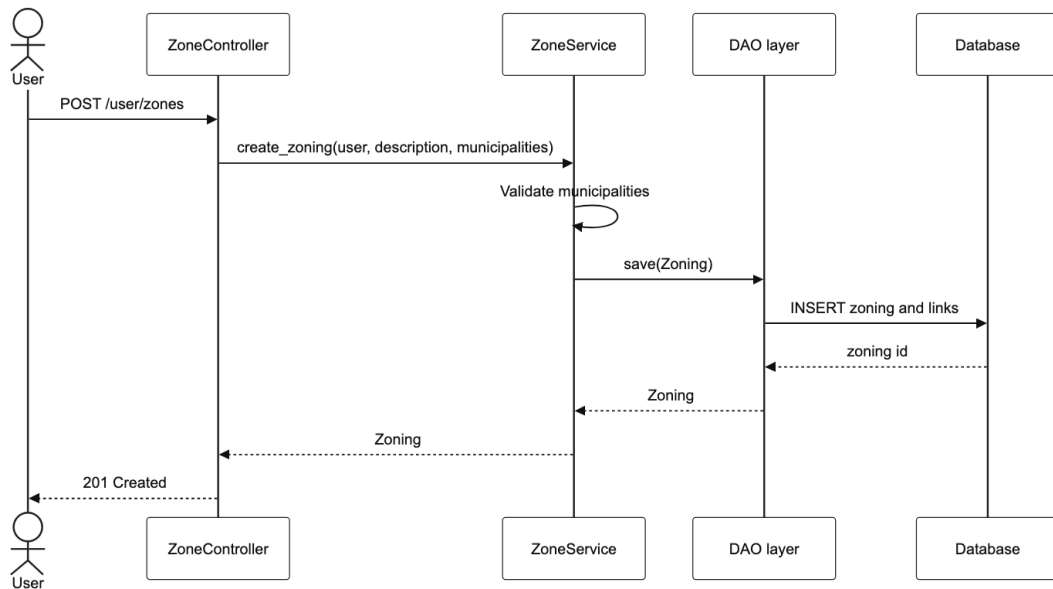


Figure A.6 : Création d'un zonage personnalisé (F4)

Le propriétaire peut ensuite modifier la description ou la composition d'un zonage, ou le supprimer.

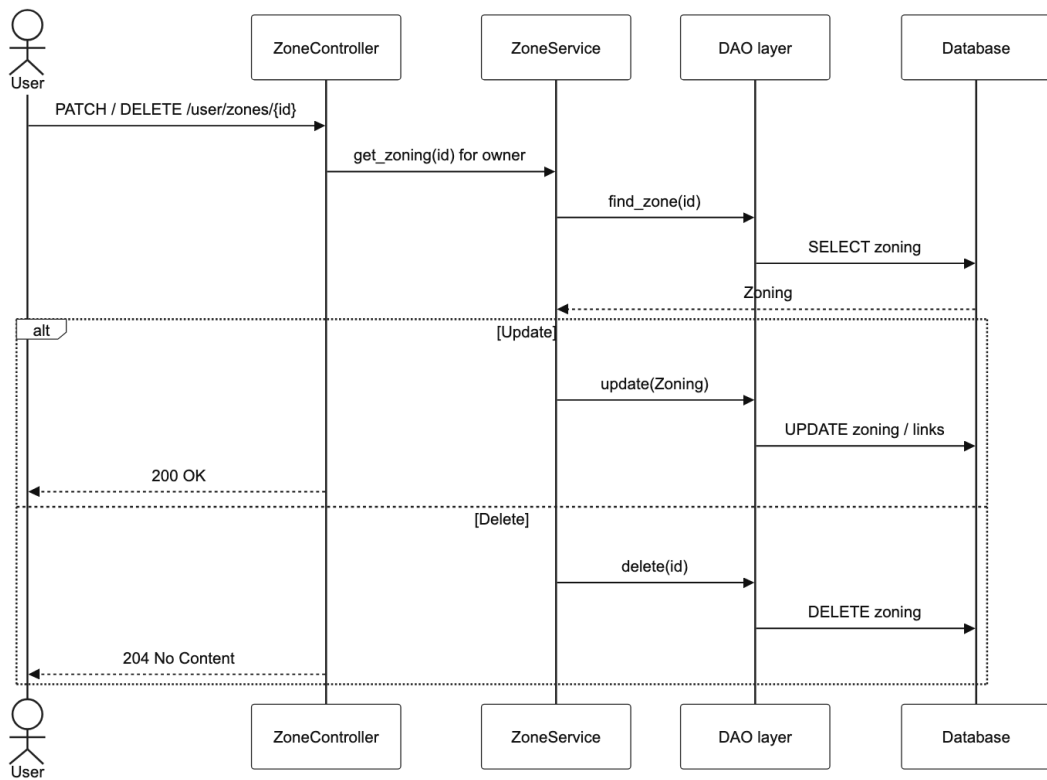


Figure A.7 : Gestion d'un zonage personnalisé (F4)

L'import optionnel (FO3) constitue un zonage à partir d'un fichier CSV ou JSON : le fichier est analysé, les communes sont résolues, puis le zonage est enregistré comme pour une création manuelle.

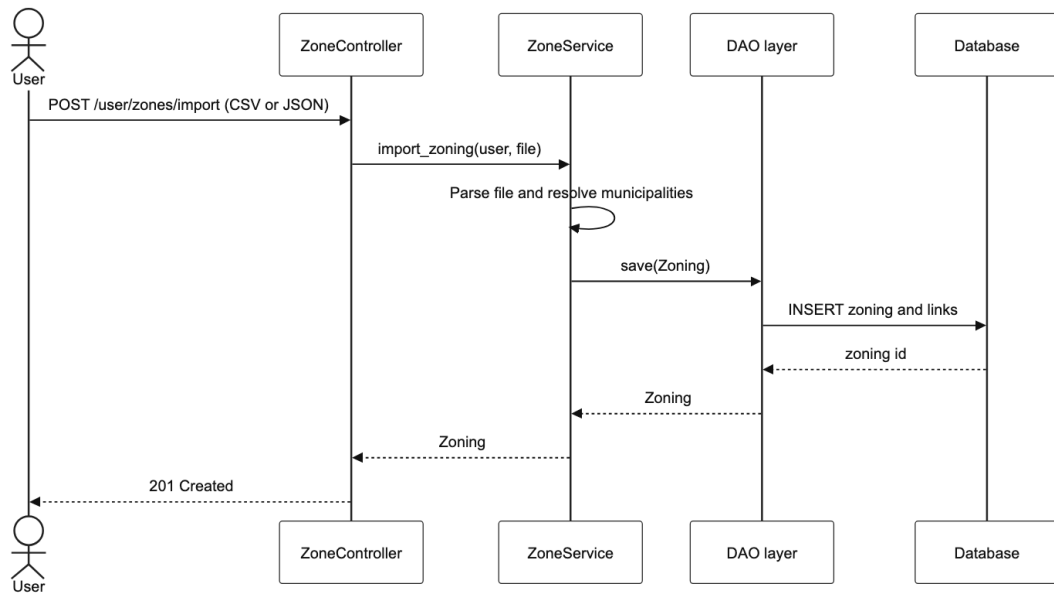


Figure A.8 : Import d'un zonage depuis un fichier (FO3)